

Module-4

Section - A Complexity Analysis of previous module

Module ① Star & Number patterns:-

All patterns below use nested loops to fill a n -row grid, so the same reasoning about loop counts applies throughout: an outer loop of n rows combined with an inner loop whose total iterations across all rows add up to a multiple of n^2 .

Star patterns

1. Square:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Two nested loops each run n times, printing one character per inner iteration, giving $n \times n = n^2$ total print operations. Only the loop counters i & j are used, so space stays constant.

2. Hollow Square:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Every one of the n^2 cells in the grid is still visited to decide whether it lies on the border (print $"^"$) or the interior (print $" "$). So the total work is still $O(n^2)$ even though most cells print blanks. No extra structure is used, so space is $O(1)$.

3. Triangle:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Row i prints i stars, so total stars printed $= 1 + 2 + \dots + n = n(n+1)/2$, which is $O(n^2)$. Space stays $O(1)$ since only the row & column counters are stored.

4. Inverted Triangle:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Row i (counting down from n to 1) prints i stars, so the total is again $n + (n-1) + \dots + 1 = n(n+1)/2 = O(n^2)$, with $O(1)$ space for the same reason as the Triangle.

5. Pyramid:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Each row prints $(n-i)$ spaces plus $(2i-1)$ stars, so every row does $O(n)$ work; across n rows this totals $O(n^2)$. No array is used, so space is $O(1)$.

6. Diamond:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

The diamond is an upper pyramid followed by a lower (inverted) pyramid, so it performs roughly twice the work of a single pyramid - still $O(n^2)$ overall, since a constant factor does no change the order of growth. Space remains $O(1)$.

7. Half diamond:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

This is a triangle immediately followed by an inverted triangle ($n-1$ rows), so the total stars printed is still a sum of consecutive integers on each half, giving $O(n^2)$ time & $O(1)$ space.

b. Half Diamond Inverted:-

Time Complexity: $O(n^2)$

Space complexity: $O(1)$

Same reasoning as half diamond, just printed top-to-bottom in the opposite order (decreasing then increasing row lengths).
: total work is still a sum of consecutive integers, giving $O(n^2)$ time and $O(1)$ space.

Number Pattern

1. Square

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Two nested loops each run n times, printing '1' once per inner iteration, giving n^2 total prints. Only loop counters are used, so space is $O(1)$.

2. Incrementing:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

for each of the n rows, the inner loop prints the values 1 through n . So n rows $\times$ n columns give n^2 total prints, with $O(1)$ space since no extra storage is used.

3. Right Triangle:-

Time complexity: $O(n^2)$

space complexity: $O(1)$

Row i prints the number 1 through i , so the total values printed is $1+2+\dots+n = \frac{n(n+1)}{2} = O(n^2)$. Space is $O(1)$, using only the row / column counters.

4. Inverted Diamond:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Row i prints the value $(i+2)$ repeated $(n-i)$ times, to the count across row again sum to a series proportional to $n^2/2$, giving $O(n^2)$ time. No extra array is used, so space is $O(1)$.

5. Sandwich Pattern:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Every one of the n^2 cells in the grid is visited once, and each cell's value is computed with a fixed no. of min(1 comparisons $O(1)$ work per cell), so the total time is $O(n^2)$. only a few counters and no extra array are used, giving $O(1)$ space.

6. Incrementing Diamond Pattern:-

Time complexity: $O(n^2)$

Space complexity: $O(1)$

Like the Star Diamond, this is an upper triangular half followed by a lower triangular half, each doing $O(n)$ work per row across n rows, for $O(n^2)$ total time & $O(1)$ space.

Module ② Searching & Sorting:

1. Fraud Transaction Detection (Linear Search)

Time complexity: $O(N)$ worst & average case $O(1)$ best case

Space complexity: $O(1)$

Linear search checks transaction IDs one by one from the start of the unsorted list. In the worst case (fraud ID is last or absent) it inspects all N entries, giving $O(N)$. It uses only a loop counter, so extra space stays constant at $O(1)$.

Optimized approach:- If the transaction IDs could be pre-sorted, binary search would reduce lookups to $O(\log N)$.

2. Server Log Search (Binary Search)

Time complexity: $O(\log N)$ all cases (Iterative) -

Space complexity: $O(1)$ iterative, $O(\log N)$ recursive (call stack).

Since log IDs are already sorted, each comparison eliminates half of the remaining search range. After K comparisons the range size is $N/2^K$, so it takes about $\log_2(N)$ steps to find out whether the target is present.

3. Sort Employee Salaries (Bubble Sort)

Time complexity: $O(N^2)$ worst & average case, $O(N)$ best case

Can use swap-flag optimization

Space complexity: $O(1)$

Bubble sort repeatedly compares adjacent salaries & swaps out of order pairs, needing $N-1$ passes over up to $N-1$ comparisons each, giving roughly N^2 operations. It sorts in place using only a temporary swap variable, so space stays $O(1)$.

Optimized approach: Merge sort or quick sort being average-case time down to $O(N \log N)$ for larger payrolls.

4. Sort website Response Time [Insertion Sort]
 Time complexity: $O(N^2)$ worst & avg. case, $O(N)$ best case (nearly sorted data)

Space complexity: $O(1)$

Each new response time is compared with, & shifted past, the already-sorted elements before it, which can take up to $N-1$ shifts per insertion in the worst case, giving $O(N^2)$. Sorting happens in-place, so only $O(1)$ extra space is needed.

Optimized approach: Since new data arrives continuously and is often nearly sorted already, Insertion Sort's best case $O(N)$ behaviour makes it a reasonable practical choice here; for large unsorted batches, Merge Sorts $O(N \log N)$ would scale better.

5. Prioritize Bug Reports (Selection Sort)

Time complexity: $O(N^2)$ in all cases

Space complexity: $O(1)$

Selection Sort scans the entire Remaining unsorted portion to find the minimum priority score in every pass, regardless of the input's initial order, so it always performs about $N^2/2$ Comparisons, giving $O(N^2)$ with no best-case improvement.

Optimized approach: Heap Sort performs the same 'repeatedly pick the minimum' idea but with a heap structure, cutting the time to $O(N \log N)$.

Module ③ Merge sort, Quick sort, Heap sort:

1. Merge sort:

Time complexity: $O(N \log N)$ in all cases (best, avg, worst)

Space complexity: $O(N)$

Merge sort repeatedly splits the array in half ($\log N$ level of recursion) and merges sorted values back together, doing $O(N)$ work at each level, for a total of $O(N \log N)$. The merge sort needs a temporary array to hold merged results, giving $O(N)$ space.

Optimized approach: Already asymptotically optimal for comparison-based sorting; its main trade-off is the extra $O(N)$ space versus in-place algorithms like quick sort or heap sort.

2. Quick sort:-

Time complexity:- $O(N \log N)$ avg. case, $O(N^2)$ worst case.

Space complexity:- $O(N \log N)$, avg. (recursion stack), $O(N)$ worst ^{case}

Quick sort partitions around a pivot & recurses on each side, when partitions split roughly in half it takes $\log N$ levels of $O(N)$ partitioning work, giving $O(N \log N)$. If the pivot is consistently the smallest or largest element, partitions become unbalanced & recursion depth grows to $O(N)$, causing the $O(N^2)$ worst case.

Optimized approach:- choosing a random pivot or the median-of-three keeps partitions balanced with very high probability, keeping performance as close to $O(N \log N)$ in practice.

3. Heap sort:-

Time complexity: $O(N \log N)$ in all cases (best, avg. worst)

Page No. _____
Date _____

Space Complexity: $O(1)$

Building the initial max-heap takes $O(N)$, and then each of the N extract-max operations takes $O(N \log N)$ to restore the heap property, giving $O(N \log N)$ overall. Since the heap is stored directly in the input array, no extra array is needed, giving $O(1)$ space.

Optimized approach: Already $O(N \log N)$ worst-case guaranteed with $O(1)$ space, which is why it is preferred over quick sort when worst-case performance matters and over merge sort when memory is limited.

Section - B Analyze the algorithm complexity.

Problem 1: Print 1 to N

C++ Code.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cout << i;
        if (i != n) cout << " ";
    }
    return 0;
}
```

Time complexity: $O(N)$ The loop runs exactly N times, & each iteration does a constant amount of work (printing 1 no.). Since the total work grows directly & linearly with N , the time complexity is $O(N)$.

Space complexity: $O(1)$ - Only a fixed no. of variables (n and the loop counter i) are used, regardless of how large N is. No array or extra data structure is created, so the space used stays constant, giving $O(1)$.

Optimized Approach: - This problem can't be solved faster than $O(N)$, because all N numbers must be individually printed at least once - there is no way to skip visiting each value.



main.cpp



Share

Run

Output

Clear

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     int n;
5     cin >> n;
6     for (int i = 1; i <= n; i++) {
7         cout << i;
8         if (i != n) cout << " ";
9     }
10    return 0;
11 }
```

```
5
1 2 3 4 5
```

=== Code Execution Successful ===

Page No. _____
Date _____

Problem - 2: Sum of first N Natural Numbers (using a loop)

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;

    long long sum = 0;
    for (long long i = 1; i <= n; i++) {
        sum += i;
    }

    cout << sum;
    return 0;
}
```

Time complexity: $O(N)$ - ^{for} The loop iterates from 1 to N, executing one addition ($sum += i$) per iteration. Since exactly a constant-time operations are performed, the overall time complexity is $O(N)$.

Space complexity: $O(1)$ - only 2 variables, n & sum are maintained throughout execution. No additional memory is allocated as N grows, so the space complexity is $O(1)$.

Optimized approach: The sum of first N Natural nos can be computed directly using the formula $N(N+1)/2$, which needs only one multiplication & one division. This reduces the time complexity from $O(N)$ to $O(1)$, since the loop is eliminated entirely.



main.cpp



Share

Run

Output

Clear

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     long long n;
5     cin >> n;
6     long long sum = 0;
7     for (long long i = 1; i <= n; i++) {
8         sum += i;
9     }
10    cout << sum;
11    return 0;
12 }
```

```
5
15

=== Code Execution Successful ===
```

3. Problem-3 A^B using a loop.

C++ code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main () {
    long long a, b;
    cin >> a >> b;

    long long result = 1;
    for (long long i = 0; i < b; i++) {
        result *= a;
    }
```

```
    cout << result;
    return 0;
}
```

Time Complexity: $O(B)$ - The loop multiplies result by a exactly B times, one multiplication per iteration. The no. of operations scales directly with the exponent B, so the time complexity is $O(B)$.

Space Complexity: $O(1)$ - The program only stores a, b, result, & the loop counter i, none of which grow with the input size. Hence the space complexity remains $O(1)$.

Optimized Approach: Fast (binary) exponentiation reduces this to $O(\log B)$; repeatedly square the base & halve the exponent, multiplying the result in only when the exponent is odd ($a^b = (a^{b/2})^2$, adjusted for odd b). This cuts the no. of multiplications from B down to about $\log_2(B)$.



C++ Online Compiler



main.cpp



Share

Run

Output

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main() {
4     long long a, b;
5     cin >> a >> b;
6     long long result = 1;
7     for (long long i = 0; i < b; i++) {
8         result *= a;
9     }
10    cout << result;
11    return 0;
12 }
```

2 3

8

=== Code Execution Successful ===